Tutorial I

Desarrollo y publicación de una aplicación web únicamente con frontend, con código fuente alojado en GitHub y despliegue automático en Microsoft Azure mediante GitHub Actions

Universidad Adolfo Ibáñez

Taller de Innovación y Emprendimiento Tecnológico

Profesor: Martín Céspedes Escuti

Ayudante Instructor: Artemio Araya

20 de octubre de 2026

Proyecto de referencia: artemioadaysolvers/Tutorial-Web-I.

Lenguajes de programación principales:

- Directamente JavaScript, HTML y CSS para Frontend
- No hay backend en este Tutorial I

Principales Herramientas sobre los Lenguajes

- three.js
- node.js

Objetivo: Explicar, paso a paso, el proceso de configuración, integración y despliegue de una aplicación web que consiste en un visualizador interactivo de mecanismos CAD convertidos a MJCF.

El presente trabajo aborda el despliegue de una arquitectura frontend en la plataforma de nube Microsoft Azure App Service. A partir del almacenamiento del código fuente en un repositorio de GitHub, se detallan los procedimientos requeridos para la automatización de los procesos de compilación, despliegue e integración automática mediante GitHub Actions.

De esta manera, se busca establecer una infraestructura en la nube que permita alojar una aplicación web funcional cuyo despliegue se actualice automáticamente ante modificaciones del código fuente en GitHub.

Índice

Definiciones 5

Estructura de una Página Web 5

Protocolos de Comunicación 6

Backend & Frontend 9

¿Por qué hay tantos lenguajes de Programación? 10

Lenguajes de programación clásicos para desarrollar páginas web 14

GitHub & GitHub Desktop para Desarrollo Web 18

GitHub 18

GitHub Pull Request 19

GitHub Actions 19

GitHub Desktop 20

¿Cómo se relaciona GitHub con Microsoft Azure para desarrollo Web para este tutorial? 21

1. Integración de GitHub con Microsoft Azure para administrar y desplegar aplicaciones web 21

Microsoft Azure 22

Windows Server 24

Servicios de Microsoft Azure útiles para desarrolo Web de este Tutorial 28

App Service 28

Tecnología que podemos desarrollar con Microsoft Azure . . . 29

Visualización Web vs. Renderizado Nativo en el Sistema Operativo 30

Descripción general del proyecto 33

Publicación de la aplicación Web 34

Funcionamiento de la web 37

Estructura del Repositorio de GitHub 41

Librerías de JavaScript Utilizadas 42

Tutorial 43

Crear la cuenta de Estudiante 45
Descargar la Aplicación Web y subir el Proyecto a GitHub 46
Crear la aplicación en Microsoft Azure 48
Nombres que deben anotarse 48
Configurar App Service 48
Configurar los permisos de Azure mediante Control de acceso (IAM) 50
Conectar Azure con GitHub Actions 52

Referencias 53

Definiciones

Estructura de una Página Web

Una página web es un documento digital que forma parte de la World Wide Web, cuyos recursos se identifican mediante direcciones URL (*Uniform Resource Locator*). Puede contener textos, imágenes y enlaces a otros documentos. Su funcionamiento se basa en una arquitectura cliente-servidor, en la que el navegador solicita los recursos a un servidor web y los interpreta para presentarlos al usuario (Tanenbaum & Wetherall, 2012).

El hipertexto permite relacionar documentos mediante enlaces, facilitando una navegación no secuencial. Para estructurar estos documentos se utiliza HTML (*HyperText Markup Language*), un lenguaje de marcado que emplea etiquetas para identificar elementos como títulos, párrafos, imágenes y enlaces (WHATWG, 2026).

La comunicación entre el navegador y el servidor se realiza mediante protocolos, que establecen las reglas para intercambiar información. En particular, HTTP (*Hypertext Transfer Protocol*) permite solicitar y recibir recursos mediante un sistema de solicitudes y respuestas. En consecuencia, el hipertexto conecta los documentos, HTML define su estructura y HTTP permite su transferencia a través de la Web (Tanenbaum & Wetherall, 2012).

Protocolos de Comunicación

Los protocolos de comunicación establecen las reglas para intercambiar información entre sistemas conectados a una red. En la Web, HTTP (*Hypertext Transfer Protocol*) permite la comunicación entre el navegador y el servidor mediante solicitudes y respuestas. El navegador solicita un recurso y el servidor responde indicando el resultado de la solicitud y entregando, cuando corresponde, el contenido requerido (Fielding et al., 2022).

Sin embargo, HTTP no incorpora por sí mismo mecanismos de cifrado, por lo que la información transmitida podría ser interceptada o modificada por terceros. Para proteger esta comunicación se utiliza HTTPS, que combina HTTP con el protocolo TLS (*Transport Layer Security*). Este último incorpora mecanismos criptográficos que permiten proteger los datos transmitidos y verificar la identidad del servidor (Rescorla, 2000, 2018).

En consecuencia, HTTPS mantiene el funcionamiento de HTTP, pero añade tres propiedades fundamentales de seguridad: confidencialidad, integridad y autenticación (Rescorla, 2018).

- **Confidencialidad**: protege el contenido mediante cifrado, de modo que su interceptación no permita conocer directamente la información transmitida.
- **Integridad**: permite detectar modificaciones no autorizadas de los datos durante la comunicación.
- **Autenticación**: permite comprobar la identidad del servidor con el que se establece la conexión. También puede utilizarse para autenticar al cliente cuando la configuración lo requiere.

En el uso habitual de HTTPS, la autenticación del servidor se apoya en certificados digitales. Estos vinculan una identidad, como un nombre de dominio, con una clave pública (Internet Security Research Group, s. f.). El navegador comprueba que el certificado corresponda al dominio solicitado, se encuentre vigente y disponga de una cadena de certificación válida hasta una autoridad de confianza (Fielding et al., 2022; Rescorla, 2018). Durante el establecimiento de la conexión, el servidor demuestra que posee la clave privada correspondiente (Fielding et al., 2022; Rescorla, 2018).

¿Quién emite los certificados digitales y cómo se determina su confiabilidad?

Los certificados digitales utilizados en HTTPS son emitidos por entidades denominadas autoridades de certificación, que verifican que el solicitante controle el dominio correspondiente. Cada certificado vincula un dominio con una clave pública y establece un periodo de validez (Internet Security Research Group, s. f.).

Cuando se accede a una página web, el navegador verifica que el certificado corresponda al dominio solicitado, esté vigente y pueda validarse mediante una cadena de firmas digitales hasta una autoridad de certificación raíz reconocida. Para ello, utiliza un almacén de certificados raíz confiables, proporcionado generalmente por el navegador o el sistema operativo, como Windows (Fielding et al., 2022; Rescorla, 2018).

En consecuencia, **un certificado válido permite autenticar al servidor y establecer una comunicación cifrada, pero no garantiza que el contenido de la página sea confiable ni que el sitio web esté libre de vulnerabilidades** (Rescorla, 2000).

¿Qué garantiza un certificado digital de confianza y cuáles son sus limitaciones?

Un certificado digital válido, utilizado en una conexión HTTPS correctamente establecida, permite autenticar al servidor y establecer una comunicación protegida mediante tres propiedades fundamentales (Fielding et al., 2022; Rescorla, 2018):

- **Autenticación**: permite verificar que el servidor está autorizado para utilizar el dominio solicitado.
- **Confidencialidad**: impide que terceros puedan leer los datos transmitidos mediante su cifrado.
- **Integridad**: permite detectar modificaciones no autorizadas de la información durante su transmisión.

Sin embargo, **un certificado válido no garantiza que el propietario del sitio sea confiable, que su contenido sea verdadero ni que la página esté libre de vulnerabilidades**. Incluso una página fraudulenta puede disponer de un certificado válido, ya que este protege la comunicación, pero no garantiza la legitimidad del sitio web (Rescorla, 2000, 2018).

Backend & Frontend

Las aplicaciones web suelen dividirse en dos componentes principales: el *frontend* y el *backend*. Esta separación permite organizar las funciones de presentación, procesamiento y almacenamiento de la información (Microsoft, 2024, 2025d).

El **frontend** corresponde a la parte de la aplicación con la que interactúa el usuario mediante el navegador. Se desarrolla principalmente utilizando HTML, CSS y JavaScript, que permiten estructurar, diseñar y proporcionar interactividad a la interfaz. A través del frontend, el usuario puede visualizar información, introducir datos y solicitar operaciones (Microsoft, 2024).

El **backend** corresponde a la parte de la aplicación que procesa las solicitudes del usuario desde el servidor. Se encarga de ejecutar las operaciones del sistema, aplicar la lógica de negocio y gestionar el acceso a las bases de datos (Microsoft, 2025d).

La comunicación entre el frontend y el backend se realiza habitualmente mediante HTTP o HTTPS, permitiendo que el navegador envíe solicitudes al servidor y reciba las respuestas correspondientes. De esta manera, el usuario puede utilizar las funciones de la aplicación sin conocer su implementación interna (Microsoft, 2025f).

Desde la perspectiva de la seguridad informática, una aplicación web debe permanecer protegida incluso si un tercero conoce el código fuente de su backend. Por ello, su seguridad debe basarse en mecanismos de autenticación, autorización y protección de credenciales, y no en mantener oculto su funcionamiento interno (Microsoft, 2023a; OWASP Foundation, s. f.).

¿Por qué hay tantos lenguajes de Programación?

Los computadores utilizan procesadores construidos mediante circuitos electrónicos que ejecutan instrucciones en lenguaje máquina, representadas mediante secuencias de bits (ceros y unos). Estas instrucciones dependen de la arquitectura del procesador, como x86-64, ARM64 o RISC-V, que utilizan diferentes conjuntos de instrucciones (Nisan, 2005).

Los programas desarrollados en lenguajes como Python, JavaScript o C++ requieren mecanismos de compilación o interpretación para ejecutarse mediante instrucciones en lenguaje máquina. El procesador interpreta estas instrucciones y realiza las operaciones correspondientes mediante sus circuitos electrónicos. En consecuencia, el lenguaje máquina constituye la interfaz entre el software y el hardware**, permitiendo ejecutar físicamente las operaciones definidas por los programas (Nisan, 2005).

Una primera razón que explica la existencia de numerosos lenguajes de programación es la evolución histórica de la informática y la necesidad de conservar sistemas desarrollados con tecnologías anteriores. Inicialmente, los programas se escribían en lenguaje máquina o ensamblador, lo que requería un considerable esfuerzo de programación. Para facilitar este trabajo surgieron lenguajes de alto nivel, como FORTRAN, desarrollado durante la década de 1950, que permitió expresar operaciones matemáticas mediante instrucciones más comprensibles (Backus, 1978).

Con el transcurso del tiempo, las empresas desarrollaron sistemas informáticos complejos utilizando los lenguajes disponibles en cada época. Muchos de estos programas continúan utilizándose, ya que reemplazarlos completamente requiere importantes inversiones en desarrollo, capacitación, pruebas y mantenimiento. Por esta razón, las organizaciones suelen conservar tecnologías antiguas y modernizar progresivamente sus sistemas (Jullien et al., 2002; Meyerovich, 2013).

Un ejemplo es el software de diseño asistido por computador EUCLID, desarrollado originalmente en Fortran. Las dificultades para mantener y ampliar su arquitectura motivaron el desarrollo de un nuevo sistema basado en C++, denominado CAS.CADE. Este proyecto involucró aproximadamente treinta desarrolladores y una inversión estimada de ocho millones de euros. **Este caso demuestra cómo la conservación de sistemas antiguos y los costos de modernización contribuyen a la coexistencia de diferentes lenguajes de programación** (Jullien et al., 2002).

En consecuencia, la permanencia de numerosos lenguajes de programación no depende únicamente de sus ventajas técnicas, sino también de factores históricos, económicos y de compatibilidad. En muchos casos, conservar sistemas existentes resulta más conveniente que reemplazarlos, lo que favorece la coexistencia de diferentes generaciones de lenguajes de programación (Jullien et al., 2002; Meyerovich, 2013).

Una segunda razón que explica la existencia de numerosos lenguajes de programación es la influencia de la formación universitaria y la experiencia de los desarrolladores. Los profesionales tienden a utilizar los lenguajes que ya conocen, incluso cuando existen alternativas más eficientes para determinadas tareas. Meyerovich y Rabkin (2013) demostraron que la experiencia previa, la disponibilidad de bibliotecas y la existencia de código desarrollado anteriormente influyen considerablemente en la elección de un lenguaje de programación.

Este fenómeno también responde a factores económicos, ya que adoptar un nuevo lenguaje puede requerir inversiones en capacitación, contratación de especialistas y adaptación de herramientas. Además, las universidades contribuyen a mantener la popularidad de determinados lenguajes. Gomes et al. (2026), en un estudio de 150 cursos de programación de 83 universidades, identificaron una concentración de la enseñanza en Python, Java, C y C++.

En consecuencia, la permanencia de los lenguajes de programación no depende únicamente de sus ventajas técnicas, sino también de la formación

de los desarrolladores, su experiencia y los costos asociados a la adopción de nuevas tecnologías (Gomes et al., 2026; Meyerovich, 2013; Shrestha et al., 2020).

Una tercera razón que explica la existencia de numerosos lenguajes de programación es la necesidad de equilibrar la facilidad de desarrollo con la eficiencia computacional. Los lenguajes de alto nivel, como Python y JavaScript, permiten desarrollar programas con instrucciones más sencillas, mientras que los lenguajes de bajo nivel, como el ensamblador, ofrecen un control más directo sobre las operaciones del procesador y la memoria. Sin embargo, ambos requieren ejecutar finalmente instrucciones en lenguaje máquina, y su eficiencia depende también de los algoritmos, compiladores y optimizaciones utilizados (Nisan, 2005; Pereira et al., 2017b).

Los lenguajes de alto nivel pueden requerir operaciones internas adicionales que aumentan el tiempo de ejecución y el consumo de recursos. Por esta razón, han surgido lenguajes especializados, denominados lenguajes de dominio específico, diseñados para facilitar la programación y generar código ejecutable eficiente en determinadas aplicaciones. No obstante, sus ventajas pueden disminuir fuera del ámbito para el cual fueron desarrollados (Mernik et al., 2005).
Un ejemplo es Halide, un lenguaje especializado en procesamiento de imágenes que permite optimizar la ejecución mediante técnicas de paralelización, vectorización y gestión de memoria. En determinados experimentos, sus autores obtuvieron programas hasta cinco veces más rápidos que implementaciones equivalentes en C optimizadas manualmente, con un menor esfuerzo de programación (Ragan-Kelley et al., 2013).

En consecuencia, la creación de nuevos lenguajes también responde a la necesidad de reducir el trabajo del desarrollador y mejorar la eficiencia computacional en aplicaciones específicas, aunque estas ventajas no necesariamente se mantienen para otros tipos de problemas (Mernik et al., 2005; Ragan-Kelley et al., 2013).

Una cuarta razón que explica la creación de nuevos lenguajes de programación es la necesidad de mejorar la seguridad informática. Lenguajes como C y C++ permiten controlar directamente la memoria del computador, pero también pueden generar vulnerabilidades cuando se utilizan incorrectamente. Por esta razón, se han desarrollado lenguajes que incorporan mecanismos de seguridad para reducir determinados errores de programación (Szekeres et al., 2013).

Un ejemplo de cómo la seguridad informática puede contribuir al reemplazo de tecnologías es el abandono de Adobe Flash, que utilizaba ActionScript y Flash Player para ejecutar aplicaciones interactivas en los navegadores. Sus problemas de seguridad, rendimiento y compatibilidad favorecieron su sustitución por tecnologías web abiertas como HTML5, JavaScript y CSS (LaForge, 2016; Adobe, 2017). JavaScript asumió numerosas funciones de programación interactiva, mientras que HTML5 incorporó herramientas para representar contenido multimedia, complementadas por CSS y WebGL para desarrollar animaciones y gráficos interactivos (WebKit Project, 2017).

En 2016, Google anunció que Chrome daría prioridad a HTML5 frente a Flash, destacando ventajas de seguridad y rendimiento (LaForge, 2016). Posteriormente, Adobe finalizó el soporte de Flash Player el 31 de diciembre de 2020 y bloqueó su ejecución desde el 12 de enero de 2021 (Adobe, 2021).
En consecuencia, las exigencias de seguridad informática pueden impulsar la creación de lenguajes más seguros y el reemplazo de tecnologías vulnerables, aunque el caso de Flash corresponde principalmente a una sustitución de plataformas y no a la creación de un nuevo lenguaje de programación (Adobe, 2021; Szekeres et al., 2013).

Lenguajes de programación clásicos para desarrollar páginas web

Las tecnologías web evolucionaron para satisfacer distintas necesidades: HTML estructura el contenido, CSS define su presentación y JavaScript incorpora interactividad (Ecma International, 2025; W3C, s. f.; WHATWG, 2026). Posteriormente, Node.js permitió utilizar JavaScript en el backend, mientras que Python y C# se consolidaron mediante frameworks como Django, Flask y ASP.NET. TypeScript surgió para facilitar el mantenimiento de aplicaciones JavaScript complejas. Cabe distinguir que HTML y CSS son lenguajes de marcado y de hojas de estilo, respectivamente, mientras que los restantes son lenguajes de programación (Ecma International, 2025; W3C, s. f.; WHATWG, 2026) (Ecma International, 2025; W3C, s. f.; WHATWG, 2026).

- **JavaScript.** Desarrollado por Brendan Eich en **1995**, permite incorporar interactividad a las páginas web. Posee tipado dinámico y se utiliza tanto en frontend como en backend mediante Node.js.
  citepwirfs

- **Python.** Desarrollado por Guido van Rossum entre **1989 y 1991**, es un lenguaje de alto nivel y tipado dinámico. Destaca por su sintaxis legible y sus aplicaciones en automatización, inteligencia artificial, análisis de datos y backend.
  citeppython-foundation

- **CSS.** Propuesto por Håkon Wium Lie en **1994**, permite definir la presentación visual de documentos HTML, incluyendo colores, tipografías y distribución de elementos. Facilita la separación entre contenido y diseño.
  citepcss-history

- **HTML.** Desarrollado por Tim Berners-Lee alrededor de **1990**, es un lenguaje de marcado que permite estructurar documentos web mediante títulos, párrafos, imágenes, enlaces y formularios.
  citepcern

- **TypeScript.** Desarrollado por Microsoft en **2012**, amplía JavaScript mediante comprobación estática de tipos, facilitando la detección de errores y el mantenimiento de aplicaciones complejas (Microsoft, 2022).

- **C#.** Desarrollado por Microsoft y anunciado en **2000**, es un lenguaje de tipado estático integrado con .NET. Se utiliza en aplicaciones empresariales, backend mediante ASP.NET Core y desarrollo de software de escritorio (Microsoft, 2000).

Tecnología	Propósito de uso actual	Creación	Uso principal en 2026	Alternativa más eficiente
JavaScript (International Digital Economy Academy, s. f.; MDN Web Docs, s. f.-b)	Desarrollar la lógica y la interactividad de aplicaciones web y servicios ejecutados en servidores.	1995	Interfaces web, aplicaciones interactivas y backend con Node.js.	**MoonBit**. Lenguaje compilado a WebAssembly que puede mejorar el rendimiento de cálculos concretos; no reemplaza las API del navegador (MoonBit, 2024).
Python (Centrum Wiskunde & Informatica, 2015; Nim Contributors, s. f.)	Automatizar tareas, analizar datos y desarrollar programas y servicios.	1989: inicio. 1991: publicación.	Inteligencia artificial, análisis de datos, automatización y backend.	**Nim**. Compila a código nativo y puede resultar más rápido en tareas intensivas en CPU; tiene un ecosistema menor (Torano, 2017).
C# (Microsoft, s. f.-o; Zig Software Foundation, s. f.)	Desarrollar aplicaciones y servicios con la plataforma .NET.	2000: anuncio. 2002: versión 1.0.	Servicios web con ASP.NET Core, aplicaciones empresariales, escritorio y videojuegos con Unity.	**Zig**. Útil para componentes nativos con control directo de memoria; puede reducir el costo de ejecución en tareas concretas, pero no sustituye el ecosistema .NET (Do, 2026; GlebOlsen, 2026).
TypeScript (Ginger Bill, s. f.; Microsoft, 2022)	Desarrollar aplicaciones basadas en JavaScript con tipos que permiten detectar errores durante el desarrollo.	2012: publicación. 2014: versión 1.0.	Interfaces con React y Next.js, aplicaciones web y backend con Node.js.	**Odin**. Genera ejecutables nativos y permite gestionar recursos directamente; puede ser útil en componentes de cálculo intensivo, pero no sustituye TypeScript en React y Next.js (MariuzM, s. f.).
HTML5 (SixtyFPS GmbH, s. f.; World Wide Web Consortium, 2014)	Estructurar documentos e interfaces web. No es un lenguaje de programación.	2004: trabajo inicial. 2014: recomendación W3C.	Estructura semántica de páginas y aplicaciones web modernas.	**Slint**. Herramienta para interfaces nativas ligeras; puede utilizar menos recursos, pero no sustituye HTML5 dentro del navegador (Slint, s. f.).

Fuente: elaboración propia con base en documentación histórica de JavaScript, Python, C#, TypeScript y HTML5 y de las alternativas citadas.

Tecnología	Ventajas frente a otras opciones	Desventajas frente a otras opciones	Uso en frontend o backend y por qué
JavaScript (Microsoft, s. f.-p; OpenJS Foundation, s. f.)	Funciona directamente en los navegadores. Frente a TypeScript, permite comenzar sin configurar la comprobación de tipos. Con Node.js puede utilizarse el mismo lenguaje en el cliente y el servidor.	Frente a TypeScript y C#, detecta menos errores de tipos antes de ejecutar. En Node.js, los cálculos largos en el hilo principal pueden retrasar otras solicitudes y requieren trabajo adicional para ejecutarse en paralelo.	Ambos. En frontend controla eventos y actualiza la interfaz del navegador. En backend, Node.js permite crear API y servicios, especialmente cuando predominan las operaciones de red y base de datos.
Python (Pereira et al., 2017a; Python Software Foundation, s. f.-a,s)	Su sintaxis facilita escribir prototipos y automatizaciones con poco código. Frente a JavaScript y C#, ofrece un ecosistema especialmente amplio para análisis de datos, cálculo científico e inteligencia artificial.	Los cálculos escritos únicamente en Python suelen ser más lentos que sus equivalentes compilados en C#. A diferencia de JavaScript, no se ejecuta directamente como lenguaje habitual del navegador; necesita herramientas adicionales.	Principalmente backend. Flask y Django permiten procesar solicitudes, gestionar usuarios y acceder a bases de datos. También facilita integrar análisis de datos e inteligencia artificial en el servidor. Puede generar HTML para el frontend.
C# (Microsoft, s. f.-d,s; Pereira et al., 2017a)	Su tipado estático detecta errores antes de ejecutar. Frente a Python, suele ofrecer mayor rendimiento en cálculos escritos en el propio lenguaje. .NET integra herramientas para API, concurrencia y aplicaciones empresariales.	Frente a Python, puede requerir más estructura y configuración para tareas pequeñas. Frente a JavaScript, no se ejecuta directamente en el navegador: para una interfaz interactiva con C# se utilizan soluciones como Blazor.	Principalmente backend; también frontend. ASP.NET Core permite desarrollar API y servicios. Blazor permite construir interfaces con C#, ejecutadas en el servidor o en el navegador mediante WebAssembly, según el modelo elegido.
TypeScript (Microsoft, s. f.-p)	Frente a JavaScript, detecta errores de tipos durante el desarrollo y facilita modificar proyectos grandes. Conserva el acceso al ecosistema JavaScript y mejora el autocompletado y la documentación del código.	Frente a JavaScript, añade configuración y aprendizaje del sistema de tipos. Sus tipos no comprueban por sí solos los datos recibidos de una API ni hacen que la aplicación se ejecute más rápido.	Ambos. En frontend se utiliza con React y Next.js para organizar interfaces con tipos. En backend se utiliza con Node.js para desarrollar API y servicios. Permite compartir definiciones de datos entre ambas partes.
HTML5 (WHATWG, 2026; World Wide Web Consortium, 2014)	Es la base estándar de las páginas web y funciona directamente en los navegadores. Frente a dibujar toda la interfaz en un lienzo gráfico, sus elementos semánticos facilitan la accesibilidad y la interpretación del contenido.	Por sí solo no implementa lógica de negocio, acceso a bases de datos ni interactividad compleja. Necesita CSS para el diseño y otros lenguajes para el comportamiento. No es una alternativa directa a los lenguajes de programación.	Frontend. Define la estructura de textos, enlaces, formularios y otros elementos que muestra el navegador. El backend puede generar HTML, pero la lógica del servidor se implementa con otro lenguaje.

Nota: comparación orientativa según las características de cada tecnología. Las ventajas dependen del proyecto, las bibliotecas y la implementación. HTML5 es un lenguaje de marcado; complementa a los lenguajes de programación.

GitHub & GitHub Desktop para Desarrollo Web

GitHub

GitHub es una plataforma en línea destinada al desarrollo, almacenamiento y administración de proyectos de software. Su funcionamiento se basa en Git, un sistema de control de versiones que permite registrar y gestionar los cambios realizados en los archivos de un proyecto. GitHub amplía estas funcionalidades mediante el alojamiento de repositorios en la nube y la incorporación de herramientas que facilitan la colaboración entre desarrolladores, la revisión del código, la planificación de tareas y la automatización de procesos de desarrollo (GitHub, s. f.-g).
Asimismo, permite mantener repositorios públicos o privados y proporciona herramientas para implementar aplicaciones y publicar sitios web. De acuerdo con su documentación oficial, GitHub está diseñado para apoyar las distintas etapas del ciclo de vida del desarrollo de software, desde la planificación inicial hasta su implementación y mantenimiento (GitHub, s. f.-g).

GitHub fue creado por Tom Preston-Werner, Chris Wanstrath y PJ Hyett en Estados Unidos. Su desarrollo comenzó en 2007 como un proyecto impulsado por Preston-Werner y Wanstrath, al que posteriormente se incorporó Hyett. La plataforma fue lanzada públicamente el 10 de abril de 2008, con el propósito de facilitar el alojamiento de repositorios de código y la colaboración entre desarrolladores mediante el sistema de control de versiones Git (Chacon, 2009; Wanstrath, 2008).

Las siguientes funcionalidades son las que utilizaremos en este tutorial.

GitHub Pull Request

GitHub Pull Requests es una funcionalidad que permite proponer, revisar y aprobar modificaciones en el código fuente antes de incorporarlas a una rama del repositorio. Facilita la comparación de versiones, la colaboración entre desarrolladores y la ejecución de pruebas automatizadas mediante GitHub Actions (GitHub, s. f.-b). Una vez cumplidos los requisitos de revisión, los cambios pueden fusionarse con la rama de destino, contribuyendo al control de calidad y al desarrollo colaborativo del software (GitHub, s. f.-b).

GitHub Actions

GitHub Actions es un servicio de automatización integrado en GitHub que permite ejecutar procesos asociados al desarrollo de software, como compilación, pruebas y despliegue de aplicaciones. Estos procesos se configuran mediante archivos YAML y pueden activarse automáticamente ante modificaciones del repositorio, mediante programación temporal o manualmente, reduciendo la ejecución de tareas repetitivas (GitHub, s. f.-i).

Su ejecución puede realizarse mediante infraestructura proporcionada por GitHub o mediante ejecutores autohospedados (GitHub, s. f.-d,s). Según la documentación oficial, los ejecutores estándar de Linux y Windows utilizan infraestructura de Microsoft Azure, mientras que los de macOS emplean infraestructura administrada por GitHub. Los ejecutores autohospedados pueden utilizar computadores propios o servicios de computación en la nube. En consecuencia, **la infraestructura utilizada depende del tipo de ejecutor y de la configuración seleccionada** (GitHub, s. f.-d,s).

GitHub Desktop

GitHub Desktop es una aplicación gratuita y de código abierto que permite administrar repositorios Git mediante una interfaz gráfica (GitHub, s. f.-a). Facilita el registro de modificaciones, la consulta del historial de versiones, la creación de ramas y la sincronización entre repositorios locales y remotos, sin necesidad de utilizar comandos en una terminal. En consecuencia, GitHub Desktop simplifica el uso de Git y su integración con la plataforma GitHub, facilitando la gestión del código fuente y el desarrollo colaborativo (GitHub, s. f.-a).

¿Cómo se relaciona GitHub con Microsoft Azure para desarrollo Web para este tutorial?

1. Integración de GitHub con Microsoft Azure para administrar y desplegar aplicaciones web

GitHub puede integrarse con Microsoft Azure para automatizar la publicación y actualización de aplicaciones web. GitHub administra el código fuente mediante repositorios, mientras que Azure App Service proporciona la infraestructura necesaria para alojar y ejecutar la aplicación. Mediante GitHub Actions es posible automatizar la compilación, las pruebas y el despliegue cuando se incorporan modificaciones al repositorio (Microsoft, 2025e).

Esta integración también admite repositorios privados, cuyo acceso se controla mediante permisos y mecanismos de autenticación, como tokens, claves SSH o GitHub Apps. Los tokens permiten autorizar operaciones específicas, como consultar un repositorio sin modificar su contenido (GitHub, s. f.-f).

Para autorizar despliegues en Azure App Service, GitHub Actions puede utilizar OpenID Connect, que permite establecer una relación de confianza mediante credenciales temporales, evitando almacenar determinadas credenciales permanentes (Microsoft, 2025e) (GitHub, s. f.-c).

En consecuencia, GitHub facilita la administración del código fuente, Azure App Service permite alojar y ejecutar la aplicación web, y GitHub Actions automatiza su despliegue. Sin embargo, mantener privado el repositorio no garantiza la seguridad de la aplicación, por lo que deben implementarse mecanismos adicionales de autenticación, autorización y protección de credenciales (Microsoft, 2023a; OWASP Foundation, s. f.).

Microsoft Azure

¿Que es Microsoft Azure?
Microsoft Azure es una plataforma de computación en la nube que proporciona servicios de procesamiento, almacenamiento, redes, bases de datos, análisis e inteligencia artificial. Permite a los desarrolladores y organizaciones crear, implementar, ejecutar y administrar aplicaciones mediante una infraestructura de servidores distribuidos globalmente, sin necesidad de mantener toda la infraestructura física de manera local (Microsoft, s. f.-k).

Fue presentada originalmente por Microsoft el 27 de octubre de 2008 bajo el nombre de Windows Azure, cuya disponibilidad comercial comenzó en febrero de 2010 (Microsoft, 2008, 2010).

La infraestructura de software de Microsoft Azure comprende diversos componentes que sustentan la ejecución y coordinación de sus servicios en la nube. Estos componentes se implementan mediante tecnologías seleccionadas de acuerdo con los requisitos técnicos de cada servicio, por lo que su desarrollo no corresponde a un único lenguaje de programación. Entre las tecnologías presentes en sus componentes públicos se encuentran el lenguaje C# y la plataforma .NET, utilizados para implementar parte de los entornos de ejecución que permiten el funcionamiento de los servicios (Microsoft, s. f.-b).

La infraestructura de Microsoft Azure se encuentra distribuida geográficamente en centros de datos (data centers) que contienen servidores físicos, sistemas de almacenamiento, de refrigeración, seguridad, infraestructura eléctrica y equipos de comunicación (Microsoft, s. f.-e,s).

Estos centros de datos se organizan en regiones geográficas interconectadas mediante redes de alta capacidad, lo que permite proporcionar servicios de computación, almacenamiento y procesamiento de información a través de Internet (Microsoft, s. f.-n).

No todos los datos de Azure se guardan en un único lugar. La región donde se almacenan y las copias adicionales que puedan existir dependen del servicio utilizado, su configuración y las políticas de residencia de datos aplicables (Microsoft, s. f.-n).

Esta permite que múltiples usuarios y organizaciones utilicen recursos informáticos sin necesidad de adquirir, instalar o administrar directamente los equipos físicos que los proporcionan.

Para comprender cómo funciona, es importante distinguir tres elementos: la infraestructura física, el software que permite operar Azure y el código fuente de las aplicaciones que desarrollan sus clientes.

Los servicios son desarrollados y mantenidos principalmente por equipos de ingeniería de Microsoft, que incluyen desarrolladores de software, especialistas en infraestructura, bases de datos, redes, ciberseguridad y operaciones.

Esto no significa que todo el software de Azure sea desarrollado exclusivamente por Microsoft. Algunos componentes utilizan proyectos de código abierto, bibliotecas externas y tecnologías mantenidas por comunidades de desarrolladores.

Microsoft mantiene gran parte del código de sus servicios e infraestructura en sistemas de desarrollo y repositorios internos con acceso restringido. La ubicación y organización exactas de todos esos repositorios no son públicas (Microsoft, s. f.-b).

Microsoft Azure utiliza diferentes sistemas operativos en su infraestructura de computación en la nube. Según Microsoft, parte de su infraestructura de virtualización se basa en versiones personalizadas y reforzadas de Windows Server, junto con tecnologías de hipervirtualización que permiten ejecutar múltiples máquinas virtuales sobre servidores físicos (Microsoft, s. f.-e).

Windows Server

¿Qué es Windows Server?

Windows Server es una familia de sistemas operativos desarrollada por Microsoft, diseñada para administrar infraestructuras informáticas, ejecutar aplicaciones empresariales y proporcionar servicios de red, almacenamiento y virtualización. A diferencia de los sistemas operativos orientados al usuario final, Windows Server incorpora herramientas especializadas para administrar recursos computacionales, usuarios y servicios en entornos empresariales y de computación en la nube (Microsoft, s. f.-m).

Asimismo, Microsoft utiliza tecnologías basadas en Linux, incluyendo Azure Linux, una distribución optimizada para determinadas cargas de trabajo y ejecución de contenedores (Microsoft, s. f.-l). Esta arquitectura permite utilizar diferentes sistemas operativos según las necesidades de los servicios y aplicaciones, sin limitar la plataforma exclusivamente a Windows o Linux (Microsoft, s. f.-l).

Característica	Linux para servidores	Windows Server	Consecuencia
Licencias y modelo de desarrollo	Familia de sistemas basada en un núcleo de código abierto. Las distribuciones pueden ser comunitarias o contar con soporte comercial. (Debian Project, 2022)	Sistema operativo comercial desarrollado por Microsoft, disponible mediante distintas ediciones y modalidades de licencia. (Microsoft, s. f.-v)	En Windows, el costo depende de núcleos, usuarios y licencias para virtualización. Linux puede evitar licencias comerciales, pero no necesariamente los gastos de soporte. (Debian Project, 2022; Microsoft, s. f.-v)
Instalación y recursos	Permite instalaciones mínimas, sin escritorio gráfico, a las que se incorporan los componentes que requiere cada servicio. (Canonical, s. f.-a)	Puede instalarse con interfaz gráfica o en modalidad reducida, denominada *Server Core*. (Microsoft, 2025g)	Una instalación mínima reduce componentes que mantener y posibles puntos de exposición, aunque ciertas funciones pueden quedar fuera. (Microsoft, 2025g)
Administración	Suele administrarse mediante terminal, acceso remoto, archivos de configuración y automatización; la interfaz gráfica es opcional. (Canonical, s. f.-c)	Permite administración gráfica, por línea de comandos y de forma remota mediante herramientas integradas. (Microsoft, 2025g)	El acceso remoto facilita operar varios servidores sin desplazarse; exige configurar conexiones seguras y dominar las herramientas de administración. (Canonical, s. f.-c; Microsoft, 2025g)
Aplicaciones y backend	Ejecuta servicios web y aplicaciones de servidor desarrolladas en diversos lenguajes y entornos de ejecución. (Canonical, s. f.-e)	También ejecuta aplicaciones web y servicios, incluidos aquellos que requieren tecnologías específicas de Windows. (Microsoft, s. f.-g, 2023b)	Los servicios multiplataforma pueden desplegarse en ambos sistemas; las dependencias exclusivas de Windows, como *.NET Framework*, pueden dificultar la migración. (Microsoft, s. f.-h, 2023b)
Contenedores	Es el entorno habitual de los contenedores Linux, que aíslan procesos y utilizan funcionalidades del núcleo Linux. (Docker, s. f.-a,s)	Permite ejecutar contenedores Windows para aplicaciones dependientes de su entorno de sistema. (Microsoft, s. f.-t)	Las imágenes deben ser compatibles con el sistema del contenedor (Docker, s. f.-b; Microsoft, s. f.-t). En AKS, las aplicaciones Windows requieren nodos de usuario Windows (Microsoft, s. f.-j).
Ciberseguridad	Dispone de separación de usuarios, permisos y mecanismos de protección configurables según la distribución. (Canonical, s. f.-d)	Ofrece autenticación, control de acceso y directivas de seguridad integradas en su ecosistema. (Microsoft, s. f.-s)	Reducir servicios y privilegios limita la exposición, pero ambos sistemas requieren parches, control de accesos y configuraciones seguras. (Canonical, s. f.-d; Microsoft, 2025g)
Identidades y usuarios	Administra usuarios y grupos locales, y puede integrarse con servicios de identidad y directorios externos. (Canonical, s. f.-b,s)	Incluye Active Directory Domain Services para centralizar identidades y recursos de una organización. (Microsoft, s. f.-a)	Un directorio central evita duplicar cuentas de usuario. Linux puede integrarse con Active Directory mediante servicios de conexión adicionales. (Canonical, s. f.-b; Microsoft, s. f.-a)
Virtualización	Puede actuar como anfitrión de máquinas virtuales mediante tecnologías como KVM. (Linux Kernel Development Community, s. f.)	Integra Hyper-V para crear y operar máquinas virtuales sobre infraestructura compatible. (Microsoft, s. f.-f)	Ambos consolidan máquinas virtuales en un solo equipo (Linux Kernel Development Community, s. f.; Microsoft, s. f.-f). En Windows, la edición licenciada condiciona los derechos de virtualización (Microsoft, s. f.-v).
Uso en Microsoft Azure	Puede utilizarse como sistema operativo de máquinas virtuales y nodos de contenedores en Azure. (Microsoft, s. f.-j,s)	También está disponible para máquinas virtuales y determinados nodos de contenedores en Azure. (Microsoft, s. f.-j,s)	Ciertas imágenes Linux agregan costos de soporte (Microsoft, s. f.-i); en Windows, *Azure Hybrid Benefit* puede reducir el gasto con licencias elegibles (Microsoft, s. f.-c).

Fuente: elaboración propia a partir de la documentación oficial de Debian, Canonical, Docker, el núcleo Linux y Microsoft. Las consecuencias se fundamentan en las fuentes citadas por fila. Comparación cualitativa, no una medición de rendimiento.

¿Qué tecnologías de programación se utilizan en la infraestructura de Microsoft Azure y que factores explican su adopción?

La infraestructura de software de Microsoft Azure no utiliza un único lenguaje de programación, ya que la plataforma está compuesta por numerosos servicios, entornos de ejecución y componentes de infraestructura que cumplen funciones diferentes. Por esta razón, no sería técnicamente correcto afirmar que todo Azure está desarrollado en C# o en cualquier otro lenguaje. Sin embargo, C# y .NET son tecnologías importantes en determinados componentes de su funcionamiento interno. Un ejemplo verificable es Azure Functions Host, el entorno de ejecución de Azure Functions, cuyo repositorio público contiene código fuente desarrollado en C# sobre .NET (Microsoft, s. f.-b).

Para comprender el origen de esta relación tecnológica, es necesario considerar los antecedentes de la plataforma. En octubre de 2008, Microsoft presentó Windows Azure, denominación original de Microsoft Azure, como una plataforma de servicios en la nube destinada a facilitar la creación, el alojamiento y la ejecución de aplicaciones. Desde su presentación, Microsoft destacó que los desarrolladores podían aprovechar conocimientos y herramientas que ya utilizaban, especialmente .NET Framework y Visual Studio, sin limitarse exclusivamente a estas tecnologías (Microsoft, 2008).

En las primeras etapas de Windows Azure, C# se empleaba frecuentemente para desarrollar la lógica de las aplicaciones, mientras que .NET Framework proporcionaba bibliotecas, servicios y un entorno de ejecución para el software escrito sobre esta plataforma. Visual Studio, por su parte, permitía escribir, organizar, compilar y depurar el código, además de configurar y preparar los paquetes destinados a su implementación en la nube. Así, los desarrolladores familiarizados con C#, ASP.NET y las herramientas de Microsoft podían trasladar parte de su experiencia al desarrollo de aplicaciones alojadas en Windows Azure (Nakashima et al., 2010).

Esta integración representó una ventaja para los desarrolladores familiarizados con el ecosistema de Microsoft, pues podían reutilizar

conocimientos, bibliotecas y herramientas en lugar de aprender desde cero un entorno de desarrollo completamente diferente. No obstante, trasladar una aplicación a la nube no siempre equivalía a copiarla sin modificaciones: en ciertos casos era necesario adaptar el manejo del almacenamiento, la comunicación entre componentes y la arquitectura de procesamiento para aprovechar correctamente la escalabilidad y la distribución de recursos de Windows Azure (Microsoft, 2008; Nakashima et al., 2010).

Con el desarrollo posterior de la plataforma, Microsoft Azure incorporó servicios y entornos de ejecución compatibles con diferentes tecnologías. Por ejemplo, Azure Functions admite el desarrollo de funciones mediante C#, Java, JavaScript, TypeScript, Python y PowerShell, entre otros mecanismos de extensibilidad; en el caso de TypeScript, su ejecución se realiza mediante su conversión a JavaScript. Esta compatibilidad indica qué lenguajes pueden utilizar los desarrolladores para crear aplicaciones en el servicio, pero **no permite deducir que todos esos lenguajes se utilicen para programar la infraestructura interna de Azure** (Microsoft, 2026a).

Servicios de Microsoft Azure útiles para desarrolo Web de este Tutorial

App Service

¿Qué es App Service?

Azure App Service es un servicio de plataforma como servicio completamente administrado, que permite alojar y ejecutar aplicaciones web, servicios de backend para aplicaciones móviles y API RESTful en la nube, sin necesidad de administrar directamente la infraestructura de servidores. Admite tecnologías como .NET, Java, Node.js, Python y PHP, tanto en entornos Windows como Linux, e incorpora funcionalidades de escalabilidad, seguridad y despliegue continuo (Microsoft, 2026b).

Tecnología que podemos desarrollar con Microsoft Azure

- **Cloud-Hosted Web API**: Interfaz que permite intercambiar información entre aplicaciones y servicios alojados en la nube mediante HTTP o HTTPS. En Microsoft Azure, puede implementarse mediante Azure App Service o Azure Functions para procesar solicitudes, ejecutar operaciones y acceder a datos de forma remota (Microsoft, 2025f, 2026b,c).

- **Aplicación Web**: Sistema de software accesible mediante un navegador, que permite al usuario interactuar con servicios y datos. Puede integrar un frontend para la presentación y un backend para el procesamiento de información. En Microsoft Azure, estas aplicaciones pueden alojarse mediante Azure App Service (Microsoft, 2026b).

- **Aplicación Móvil**: Software desarrollado para dispositivos Android e iOS que permite ejecutar funcionalidades locales e incorporar servicios en la nube para autenticación, almacenamiento, sincronización de datos y notificaciones (Microsoft, s. f.).

Visualización Web vs. Renderizado Nativo en el Sistema Operativo

Las interfaces gráficas permiten visualizar información e interactuar con las aplicaciones mediante herramientas propias de cada sistema operativo. La siguiente tabla presenta las principales tecnologías utilizadas en Windows, macOS, Linux y Android, junto con una comparación del consumo de memoria RAM, procesador y energía entre aplicaciones nativas y web. Los resultados dependen de las tecnologías y aplicaciones evaluadas, por lo que no representan ventajas generales de cada sistema operativo (Apple, s. f.-b; Google, s. f.; GTK Development Team, s. f.; Horn et al., 2023; Metri et al., 2015; Microsoft, s. f.-u; Milichiewicz & Badurowicz, 2025).

Sistema operativo	Lenguajes y herramientas nativas	Comparación conceptual	Memoria RAM	Procesador (CPU)	Consumo energético	Fluidez y animaciones	Calidad visual
Windows	C#, C++. WinUI, WPF, Windows Forms y Win32 (Microsoft, s. f.-u).	Algunas aplicaciones de escritorio consumen menos recursos que sus alternativas web. El rendimiento depende de su implementación (Metri et al., 2015; Milichiewicz & Badurowicz, 2025).	Qt/Python presentó menor consumo de RAM que Electron. El resultado no puede generalizarse a todas las aplicaciones (Milichiewicz & Badurowicz, 2025).	No se demostró una ventaja general. El consumo depende de las tareas ejecutadas (Metri et al., 2015; Milichiewicz & Badurowicz, 2025).	Algunas aplicaciones instaladas consumieron menos energía que sus equivalentes web (Metri et al., 2015).	No se demostró una ventaja general de las aplicaciones nativas (Metri et al., 2015; Milichiewicz & Badurowicz, 2025).	No se estableció una superioridad general en calidad gráfica (Metri et al., 2015; Milichiewicz & Badurowicz, 2025).
macOS	Swift, Objective-C. SwiftUI y AppKit (Apple, s. f.-a,s).	El rendimiento depende del framework utilizado. La comparación disponible entre Qt/Python y Electron no evalúa directamente SwiftUI (Milichiewicz & Badurowicz, 2025).	Qt/Python consumió menos RAM que Electron en el estudio. No constituye una comparación directa de SwiftUI (Milichiewicz & Badurowicz, 2025).	No se estableció una ventaja general en utilización de CPU (Milichiewicz & Badurowicz, 2025).	No se estableció una ventaja energética de las interfaces nativas de macOS (Milichiewicz & Badurowicz, 2025).	No se demostró una mayor fluidez de SwiftUI o AppKit (Milichiewicz & Badurowicz, 2025).	No se evaluó una superioridad general en calidad visual (Milichiewicz & Badurowicz, 2025).
Linux	C, C++, Python, entre otros. GTK y Qt (GTK Development Team, s. f.; The Qt Company, s. f.).	El consumo de recursos depende del framework y de la implementación (Milichiewicz & Badurowicz, 2025).	Qt/Python utilizó menos RAM que Electron. El resultado no representa a todas las aplicaciones de Linux (Milichiewicz & Badurowicz, 2025).	No se demostró una reducción general del consumo de CPU (Milichiewicz & Badurowicz, 2025).	No se estableció una ventaja energética general de GTK o Qt (Milichiewicz & Badurowicz, 2025).	No se demostró una mayor fluidez general de las interfaces nativas (Milichiewicz & Badurowicz, 2025).	No se estableció una superioridad general en calidad gráfica (Milichiewicz & Badurowicz, 2025).
Android	Kotlin, Java. Jetpack Compose y Android Views (Google, s. f.).	Las aplicaciones nativas presentaron mayor eficiencia en el uso de recursos que sus versiones web analizadas (Horn et al., 2023).	Las aplicaciones nativas consumieron menos RAM que sus versiones web (Horn et al., 2023).	Las aplicaciones nativas presentaron menor utilización de CPU en la comparación (Horn et al., 2023).	Las aplicaciones nativas consumieron menos energía, favoreciendo la autonomía del dispositivo (Horn et al., 2023).	No se demostró una ventaja general en fluidez o animaciones (Horn et al., 2023).	No se evaluó una superioridad en calidad visual (Horn et al., 2023).

Fuente: Elaboración propia a partir de (Apple, s. f.-a,s; Google, s. f.; GTK Development Team, s. f.; Horn et al., 2023; Metri et al., 2015; Microsoft, s. f.-u; Milichiewicz & Badurowicz, 2025; The Qt Company, s. f.).

¿Por qué se utilizan tecnologías web en lugar de interfaces nativas, si estas pueden consumir menos recursos?

Una de las principales ventajas de HTML, CSS y JavaScript es que permiten desarrollar aplicaciones compatibles con diferentes sistemas operativos mediante una misma base de código. En cambio, utilizar exclusivamente tecnologías nativas puede requerir desarrollar y mantener versiones específicas para cada plataforma. Los navegadores permiten ejecutar las interfaces web en distintos dispositivos, reduciendo el trabajo de desarrollo y facilitando su distribución. Por tanto, aunque algunas aplicaciones nativas presentan un menor consumo de recursos, las tecnologías web ofrecen ventajas importantes en compatibilidad, portabilidad y mantenimiento (MDN Web Docs, s. f.-c).

Descripción general del proyecto

La aplicación web, presentada como Cuaderno de mecanismos, permite explorar dos ensamblajes mecánicos tridimensionales desde el navegador. La página principal reúne una pinza de engranajes y una pinza articulada, junto con las referencias de sus modelos CAD y los créditos de las herramientas utilizadas. Cada modelo se muestra mediante un visualizador AutoMind integrado en la página (artemioadaysolvers, s. f.-b).

La interfaz utiliza HTML, CSS y JavaScript. Los archivos de los visualizadores contienen la descripción de los modelos en formato MJCF y sus recursos embebidos, mientras que Three.js proporciona las herramientas de representación gráfica tridimensional. La interacción con la escena se ejecuta en el navegador; Azure App Service ejecuta un servidor Node.js que entrega los archivos del sitio al visitante (artemioadaysolvers, s. f.-b; Microsoft, 2026b; Three.js, s. f.).

Publicación de la aplicación Web

Etapa 1. GitHub

Almacenamiento del código fuente de la aplicación
Archivos HTML, CSS y JavaScript en la carpeta site/
Servidor Node.js y configuración de la publicación automática

↓

Etapa 2. GitHub Actions

Detecta los cambios enviados a la rama main
Comprueba los archivos, valida Node.js y se autentica en Azure
Ejecuta las instrucciones de publicación

↓

Etapa 3. Transferencia de archivos

La acción azure/webapps-deploy@v3 transfiere
la aplicación completa, incluida site/, a Azure App Service
Valida Node.js sin compilar el frontend ni usar Docker

↓

Etapa 4. Azure App Service

Ejecuta el servidor Node.js de la aplicación web
Permite acceder a ella mediante una dirección pública
Entrega los archivos al navegador utilizando HTTPS

↓

Etapa 5. Navegador del usuario

Interpreta los archivos HTML y CSS
Ejecuta JavaScript para controlar la interactividad
Representa los dos mecanismos 3D mediante WebGL

(artemioadaysolvers, s. f.-b; GitHub, s. f.-i; Microsoft, 2025b, 2026b)

Etapa 1. GitHub: Almacenamiento del código fuente. GitHub almacena los archivos necesarios para desarrollar y publicar la aplicación web. En este proyecto, la carpeta site/ contiene el archivo index.html, que define la página principal, las hojas de estilo CSS y dos visualizadores tridimensionales AutoMind desarrollados mediante HTML y JavaScript. También contiene server.js, package.json y package-lock.json, que permiten ejecutarla mediante Node.js. Además, el repositorio contiene un archivo de configuración en .github/workflows/, que establece las instrucciones necesarias para automatizar la publicación de la aplicación en Microsoft Azure (artemioadaysolvers, s. f.-b).

Etapa 2. GitHub Actions: Automatización de la publicación. GitHub Actions es un servicio de GitHub que permite ejecutar automáticamente instrucciones cuando ocurren determinados eventos en un repositorio. En este proyecto, el proceso se activa cuando se envían modificaciones a la rama main, aunque también puede iniciarse manualmente. GitHub Actions utiliza un entorno de ejecución basado en Ubuntu para descargar los archivos del repositorio, comprobar que exista la página principal y validar Node.js mediante npm ci y npm run build, antes de iniciar sesión en Microsoft Azure. La autenticación se realiza mediante OpenID Connect, utilizando las credenciales de configuración almacenadas como secretos del repositorio. Este procedimiento autoriza a GitHub Actions a publicar la aplicación. (artemioadaysolvers, s. f.-b; GitHub, s. f.-i; Microsoft, s. f.-q).

Etapa 3. Transferencia de los archivos a Microsoft Azure. Una vez establecida la conexión con Azure, GitHub Actions ejecuta la acción azure/webapps-deploy@v3, que permite transferir el proyecto completo, incluida la carpeta site/, a la aplicación de Azure App Service identificada en el grupo rg-visualizador-web, por ejemplo, visualizador-web-estudiante01, en su entorno de producción (Production). Como la interfaz está desarrollada directamente con HTML, CSS y JavaScript, no es necesario compilar el frontend ni construir una imagen Docker. GitHub Actions ejecuta npm ci y npm run build para validar el proyecto Node.js; este último comando comprueba la sintaxis de server.js. Por lo tanto, la publicación consiste principalmente en transferir los archivos existentes al servicio que los ejecutará y alojará (artemioadaysolvers, s. f.-b; Microsoft, 2025b).

Etapa 4. Azure App Service: Alojamiento de la aplicación web. Azure App Service es el servicio de Microsoft Azure encargado de alojar la aplicación y permitir su acceso a través de Internet. Una vez publicados los archivos, Azure App Service ejecuta el servidor Node.js definido en server.js, que responde a las solicitudes de los navegadores y entrega los documentos HTML, las hojas de estilo CSS y los archivos JavaScript a través de HTTPS. De esta manera, los usuarios pueden acceder a la aplicación utilizando una dirección web pública, sin necesidad de descargar manualmente los archivos del repositorio de GitHub (artemioadaysolvers, s. f.-b; Microsoft, 2026b).

Etapa 5. Navegador: Ejecución de la interfaz interactiva. Cuando un usuario accede a la dirección pública de la aplicación, su navegador solicita y recibe los archivos proporcionados por Azure App Service. El navegador interpreta HTML para construir la estructura de la página, aplica CSS para establecer su apariencia y ejecuta JavaScript para incorporar interactividad. La página principal contiene dos visualizadores AutoMind, integrados mediante elementos iframe, que permiten explorar modelos de mecanismos mecánicos tridimensionales. Estos visualizadores utilizan JavaScript y WebGL para representar los modelos 3D y permitir operaciones como rotar la vista, acercar la cámara e inspeccionar los componentes. El procesamiento gráfico interactivo se realiza principalmente en el dispositivo del usuario y no requiere que Azure ejecute las simulaciones gráficas (artemioadaysolvers, s. f.-b).

En consecuencia, GitHub se encarga de almacenar el código fuente, GitHub Actions automatiza su publicación, Azure App Service aloja los archivos y el navegador del usuario ejecuta la interfaz interactiva. Esta arquitectura permite actualizar la aplicación modificando los archivos del repositorio y enviando los cambios a la rama main, sin necesidad de transferirlos manualmente a Azure ni desarrollar un backend de procesamiento para el funcionamiento de los visualizadores (artemioadaysolvers, s. f.-b; GitHub, s. f.-i; Microsoft, 2026b).

Funcionamiento de la web

Etapa 1. Acceso a la aplicación web

El usuario ingresa a la dirección HTTPS de la página
Azure App Service los entrega mediante Node.js
El navegador comienza a interpretar su contenido

↓

Etapa 2. Construcción de la página principal

HTML define la estructura y CSS su apariencia
La página incorpora dos visualizadores mediante iframe
Cada iframe carga un documento HTML independiente

↓

Etapa 3. Carga de los modelos tridimensionales

Los HTML contienen los modelos en formato MJCF
Incluyen geometrías, texturas y otros recursos
También incorporan la biblioteca Three.js

↓

Etapa 4. Carga del sistema AutoMind

GitHub API identifica la última versión del código
jsDelivr proporciona los módulos JavaScript de AutoMind
El navegador descarga y ejecuta estos módulos

↓

Etapa 5. Representación tridimensional

AutoMind interpreta el modelo MJCF y prepara los controles
Three.js organiza la escena, las cámaras y los materiales
WebGL permite representar los modelos 3D en el navegador

↓

Etapa 6. Interacción del usuario

Rotación, acercamiento y desplazamiento de la cámara
Selección e inspección de componentes
Herramientas de despiece y cortes del modelo

(artemioadaysolvers, s. f.-a,s; Khronos Group, s. f.; Three.js, s. f.)

Etapa 1. Acceso a la aplicación web. Cuando un usuario ingresa a la dirección pública de la aplicación, su navegador envía una solicitud HTTPS a Azure App Service. Su servidor Node.js responde proporcionando los archivos necesarios para presentar la página web, incluyendo documentos HTML, hojas de estilo CSS y código JavaScript. El navegador recibe estos recursos y comienza a procesarlos para construir la interfaz que visualizará el usuario. En esta arquitectura, Azure se encarga de distribuir los archivos, mientras que el procesamiento de la interfaz se realiza principalmente en el navegador (artemioadaysolvers, s. f.-b; Microsoft, 2026b).

Etapa 2. Construcción de la página principal. El navegador interpreta el archivo site/index.html, que define la estructura y el contenido principal de la página. Posteriormente, aplica las reglas de presentación establecidas en site/assets/styles.css, que determinan los colores, las tipografías y la distribución de los elementos. La página presenta una apariencia de cuaderno de mecanismos e incorpora dos visualizadores tridimensionales mediante elementos HTML iframe. Estos elementos permiten mostrar documentos HTML independientes dentro de la misma página, de modo que cada visualizador puede ejecutar sus propios programas JavaScript (artemioadaysolvers, s. f.-b).

Etapa 3. Carga de los modelos tridimensionales. Cada visualizador carga un archivo HTML ubicado en la carpeta site/viewers/. El primero contiene el mecanismo Robot_Gripper, compuesto por 20 componentes, mientras que el segundo incorpora el mecanismo 262_GRIPPER_4, compuesto por 37 componentes. Los modelos se encuentran descritos mediante MJCF (*MuJoCo XML Format*), un formato que permite representar la estructura de mecanismos, sus cuerpos, articulaciones y propiedades geométricas. Además, los archivos HTML incluyen las geometrías, texturas y recursos gráficos necesarios para representar los modelos. También contienen una copia de Three.js, biblioteca de JavaScript utilizada para trabajar con gráficos tridimensionales (artemioadaysolvers, s. f.-b; Three.js, s. f.).

Etapa 4. Carga del sistema AutoMind. Aunque los modelos tridimensionales y la biblioteca Three.js están incorporados en los archivos HTML, parte del código responsable del funcionamiento de los visualizadores pertenece

al proyecto AutoMind y se obtiene desde Internet. Para ello, el navegador consulta la API de GitHub y obtiene el identificador SHA del último commit de la rama main del repositorio artemioadaysolvers/AutoMind-URDF-USD-MJCF-Loader. Posteriormente, utiliza este identificador para solicitar a jsDelivr los módulos JavaScript correspondientes a esa versión de AutoMind. Este procedimiento permite cargar el código actualizado del visualizador sin necesidad de volver a publicar los archivos de la página en Azure (artemioadaysolvers, s. f.-a,s).

Etapa 5. Representación tridimensional de los mecanismos. Una vez cargados los módulos de AutoMind, el navegador ejecuta las instrucciones necesarias para interpretar la descripción MJCF de cada mecanismo y organizar sus componentes. AutoMind prepara las herramientas de interacción, mientras que Three.js administra los objetos tridimensionales, materiales, luces, cámaras y demás elementos de la escena. Para representar gráficamente estos objetos, Three.js utiliza WebGL, una interfaz de programación que permite ejecutar operaciones gráficas mediante las capacidades de aceleración del dispositivo del usuario. De esta manera, los mecanismos tridimensionales se representan directamente en el navegador, sin necesidad de que Azure App Service genere continuamente las imágenes de los modelos (artemioadaysolvers, s. f.-a; Khronos Group, s. f.; Three.js, s. f.).

Etapa 6. Interacción del usuario con los modelos tridimensionales. Una vez representados los mecanismos, el usuario puede interactuar con ellos mediante las herramientas de AutoMind. Estas permiten modificar la orientación y la posición de la cámara, acercar o alejar la vista, seleccionar componentes e inspeccionar la estructura de los ensamblajes. Además, los visualizadores incorporan herramientas para realizar despieces y cortes que facilitan la observación de los elementos internos y las relaciones geométricas entre las piezas. Los controles también pueden reproducir sonidos de activación mediante un archivo de audio WAV incorporado en los documentos HTML. Estas operaciones se realizan principalmente mediante JavaScript en el navegador del usuario (artemioadaysolvers, s. f.-a,s).

En conjunto, el funcionamiento de la aplicación se basa en la distribución de archivos web desde Azure App Service y su ejecución en el navegador del usuario. HTML y CSS permiten construir la interfaz, mientras que JavaScript, AutoMind, Three.js y WebGL hacen posible la representación y manipulación interactiva de los modelos mecánicos. Aunque las geometrías, texturas y otros recursos se encuentran incorporados en los archivos publicados, el sistema requiere conexión a Internet para descargar los módulos remotos de AutoMind. Esta arquitectura permite ofrecer una aplicación web tridimensional interactiva sin desarrollar un backend de procesamiento ni incorporar una base de datos (artemioadaysolvers, s. f.-a,s; Khronos Group, s. f.; Three.js, s. f.).

Estructura del Repositorio de GitHub

```
Tutorial-Web-I/
|-- .gitattributes          # Configuracion de atributos de Git
|-- .gitignore              # Archivos excluidos de Git
|-- .github/                # Configuracion de GitHub
|   `-- workflows/          # Automatizacion con GitHub Actions
|       `-- deploy-azure-existing.yml
|                            # Despliegue automatico en Azure
|-- ATTRIBUTIONS.md         # Creditos y atribuciones del proyecto
|-- FONT-LICENSE.txt        # Licencia de las fuentes tipograficas
|-- README.md               # Documentacion del proyecto
|-- Tutoria_Web__I.pdf      # Tutorial de la aplicacion
|-- package.json            # Configuracion de Node.js
|-- package-lock.json       # Registro de dependencias npm
|-- server.js               # Servidor web con Node.js
`-- site/                   # Archivos servidos desde Azure
    |-- index.html          # Pagina principal de la web
    |-- assets/             # Recursos de presentacion
    |   `-- styles.css      # Colores, fuentes y estilos CSS
    `-- viewers/            # Visualizadores 3D AutoMind
        |-- AutoMind_MJCF_Modular_Viewer_1.html
        |                    # Visualizador Robot_Gripper
        `-- AutoMind_MJCF_Modular_Viewer_2.html
                             # Visualizador 262_GRIPPER_4
```

Librerías de JavaScript Utilizadas

La página principal utiliza HTML, CSS y JavaScript sin un framework de interfaz. La representación tridimensional se apoya en Three.js y en los módulos de AutoMind. GitHub Actions ejecuta npm ci y npm run build para validar el servidor Node.js, sin requerir bibliotecas npm externas adicionales (artemioadaysolvers, s. f.-b).

- **Three.js** Biblioteca de JavaScript que proporciona clases y herramientas para construir escenas tridimensionales, organizar geometrías y materiales, configurar cámaras y luces y representar el resultado mediante WebGL. Los dos HTML incluyen una copia embebida de la revisión r132, disponible mediante globalThis.THREE. El código AutoMind utiliza ese objeto para construir y mostrar los modelos. La licencia MIT de Three.js corresponde a esa biblioteca y no asigna la misma licencia a los diseños CAD (artemioadaysolvers, s. f.-b; Three.js, s. f.).

- **Sistema modular AutoMind para MJCF:** Conjunto de módulos JavaScript del repositorio AutoMind-URDF-USD-MJCF-Loader. Integra la interpretación del modelo, los recursos embebidos, la escena y los controles del visualizador. Su entrada es MJCF_Viewer/mjcf_viewer_main.js. Los HTML consultan el último commit de main y lo utilizan para cargar este código desde jsDelivr; por ello no corresponde atribuirle una versión fija únicamente a partir del nombre del archivo HTML (artemioadaysolvers, s. f.-a,s).

Tutorial

Este tutorial explica cómo publicar y actualizar una página web con dos visualizadores de modelos mecánicos utilizando GitHub, GitHub Actions y Azure App Service. El procedimiento parte del repositorio Tutorial-Web-I y de una aplicación de App Service previamente configurada para funcionar en Linux con un entorno de ejecución PHP.

Linux es el sistema operativo del entorno que aloja la aplicación. La configuración PHP proporciona un entorno de alojamiento cuyo servidor web también puede entregar archivos HTML, CSS y JavaScript (Microsoft, 2025a).

En este proyecto, esos archivos contienen la página y sus visualizadores: el navegador los descarga y ejecuta el código JavaScript necesario para mostrar los modelos e interactuar con ellos (artemioadaysolvers, s. f.-b; Microsoft, 2026b).

GitHub conserva los archivos del proyecto y GitHub Actions automatiza su publicación. Cuando se envían cambios a la rama main, se ejecuta el flujo que obtiene los archivos del repositorio, se autentica en Azure y publica el contenido actualizado en App Service (artemioadaysolvers, s. f.-b; GitHub, s. f.-i; Microsoft, 2025b).

Se conserva el mecanismo de autenticación previamente configurado para que GitHub Actions tenga autorización para realizar ese despliegue. Esta autenticación identifica al proceso de publicación ante Azure y le permite actualizar los archivos de la aplicación (artemioadaysolvers, s. f.-b; Microsoft, s. f.-q).

La carpeta seleccionada para publicar es site/. Contiene la página principal index.html, los estilos de assets/ y los dos visualizadores de viewers/. El flujo envía el contenido de esta carpeta a App Service, manteniendo su organización y sus rutas relativas, para que el servidor pueda entregarlo a los visitantes (artemioadaysolvers, s. f.-b; Microsoft,

2025b).

Crear la cuenta de Estudiante

1. Entra en Azure for Students, selecciona **Comenzar gratis** y completa la verificación académica.
2. Abre portal.azure.com e inicia sesión.
3. En **Suscripciones**, comprueba que tienes seleccionada la suscripción correcta, por ejemplo **Azure for Students**.

Descargar la Aplicación Web y subir el Proyecto a GitHub

Aplicación Web: artemioadaysolvers/Tutorial-Web-I.

Para descargar la aplicación web y subir sus archivos a un repositorio propio de GitHub, se utilizará **GitHub Desktop**, una aplicación que permite trabajar con repositorios Git mediante una interfaz gráfica, sin necesidad de ejecutar comandos desde una terminal (GitHub, s. f.-e).

1. Descargar e instalar **GitHub Desktop** desde https://desktop.github.com/. Una vez instalada la aplicación, iniciar sesión con una cuenta de GitHub.
2. Crear un repositorio para almacenar el proyecto. Existen dos alternativas:
   - **Crear un repositorio desde GitHub Desktop:** seleccionar **File** → **New Repository**, indicar el nombre del repositorio y su ubicación local. Posteriormente, seleccionar **Publish repository** para publicarlo en GitHub.
   - **Clonar un repositorio creado previamente desde la página web de GitHub:** si el repositorio ya fue creado manualmente en GitHub, seleccionar **File** → **Clone Repository**, elegir el repositorio vacío y presionar **Clone**. Esto creará una copia local del repositorio en el computador.
3. En GitHub Desktop, seleccionar el repositorio de destino y abrir **Repository** → **Show in Explorer**. Esto abrirá en el Explorador de archivos de Windows la carpeta local asociada al repositorio.
4. Acceder al repositorio de la aplicación web: artemioadaysolvers/Tutorial-Web-I. Seleccionar **Code** → **Local** → **Download ZIP** para descargar el proyecto completo en un archivo comprimido.
5. Descomprimir el archivo ZIP descargado. A continuación, copiar todos los archivos y carpetas contenidos en el proyecto y pegarlos directamente en la carpeta local del repositorio abierta anteriormente mediante **Show in Explorer**. Se debe conservar la estructura original

del proyecto y evitar introducir una carpeta adicional que contenga todos sus archivos.

6. Regresar a **GitHub Desktop**. En la pestaña **Changes** aparecerán automáticamente los archivos incorporados al repositorio. En el campo **Summary**, escribir una descripción de los cambios, por ejemplo, *Carga inicial de la aplicación web*. Posteriormente, presionar **Commit to main** para registrar los cambios en el repositorio local.
7. Finalmente, seleccionar **Push origin** para enviar los cambios registrados al repositorio remoto alojado en GitHub. Una vez finalizada la operación, abrir el repositorio desde la página web de GitHub y comprobar que todos los archivos y carpetas se encuentren correctamente publicados (GitHub, s. f.-e).

Es importante distinguir entre **Commit to main**, que registra los cambios en el repositorio local del computador, y **Push origin**, que envía dichos cambios al repositorio remoto de GitHub. Por el contrario, **Pull origin** permite incorporar al repositorio local los cambios disponibles en el repositorio remoto (GitHub, s. f.-e).

Crear la aplicación en Microsoft Azure

Nombres que deben anotarse

Los siguientes nombres identifican los recursos del ejemplo. El nombre de la aplicación debe estar disponible; puede ser necesario cambiar su sufijo (artemioadaysolvers, s. f.-b; Microsoft, 2025c).

Elemento	Ejemplo
Repositorio GitHub	Tutorial-Web-I
Grupo de recursos	rg-visualizador-web
Plan de App Service	plan-visualizador-web
Aplicación web	visualizador-web-estudiante01
Rama de publicación	main
Carpeta publicada	site/

Configurar App Service

Buscar **App Services** en Azure Portal y seleccionar **Crear** → **Aplicación web**. La selección de Código, una pila Node - 26 y Linux corresponde al entorno del ejercicio (artemioadaysolvers, s. f.-b; Microsoft, 2025c).

- Elegir la suscripción y crear el grupo rg-visualizador-web.
- Escribir el nombre disponible para la aplicación. Por ejemplo, visualizador-web-estudiante01.
- En **Publicar**, seleccionar **Código**.
- En **Pila del entorno**, seleccionar **Node - 26**, si sigue disponible y soportado.
- Mantener **Linux** como sistema operativo.
- Elegir **Chile Central**, si la suscripción y el plan permiten esa región.

- Crear el plan plan-visualizador-web y seleccionar **F1 / Gratis** cuando esté disponible.
- Pulsar **Revisar y crear**, comprobar el costo estimado y seleccionar **Crear**. Al terminar, pulsar **Ir al recurso**.

La disponibilidad de F1 se comprueba para la región elegida. Seleccionar Node - 26 no cambia los lenguajes del proyecto (artemioadaysolvers, s. f.-b; Microsoft, 2025a, 2026d).

Configurar los permisos de Azure mediante Control de acceso (IAM)

1. En Azure Portal, abrir **Grupos de recursos** y seleccionar rg-visualizador-web. Entrar en **Control de acceso (IAM)** y seleccionar **Comprobar acceso** → **Ver mi acceso**.
2. Comprobar que la cuenta tenga el rol **Propietario (Owner)** sobre el grupo de recursos. Este rol proporciona los permisos necesarios para crear identidades administradas y asignarles roles. Si ya dispone de estos permisos, no es necesario modificar nada.
3. Si la cuenta no dispone de los permisos necesarios, solicitar a un administrador autorizado que los conceda. Para la configuración automática se requieren, entre otros, los siguientes permisos:

   - Microsoft.Authorization/roleAssignments/write
   - Microsoft.ManagedIdentity/userAssignedIdentities/write

   Estos permisos permiten crear la identidad administrada y configurar sus autorizaciones para el despliegue.
4. En Azure Portal, abrir **App Services** y seleccionar la aplicación web visualizador-web-estudiante01. En el menú izquierdo, entrar en **Implementación** → **Centro de implementación**.
5. En **Origen**, seleccionar **GitHub** y mantener **GitHub Actions** como proveedor de compilación y despliegue.
6. Iniciar sesión en GitHub y autorizar el acceso de Azure al repositorio. La cuenta de GitHub debe disponer de permisos de escritura sobre el repositorio.
7. Seleccionar la organización o usuario de GitHub, el repositorio Tutorial-Web-I y la rama main.
8. En **Opción de flujo de trabajo**, seleccionar **Agregar flujo de trabajo**, si aparece esta opción, para que Azure genere automáticamente un nuevo archivo YAML. Si el repositorio contiene un workflow anterior de despliegue, eliminarlo o desactivarlo previamente para evitar ejecuciones duplicadas.
9. Si se solicita la pila tecnológica y su versión, seleccionar **Node.js 26**, de acuerdo con la configuración de la aplicación web.

10. En **Tipo de autenticación**, seleccionar **Identidad asignada por el usuario (User-assigned identity)**. Elegir la opción de crear una nueva identidad administrada propuesta por Azure, si está disponible.
11. Azure configurará la identidad administrada y su autorización para publicar en App Service mediante **OpenID Connect (OIDC)**. No será necesario crear manualmente las credenciales de autenticación.
12. Pulsar **Guardar**. Azure generará el archivo YAML del workflow en el repositorio de GitHub, dentro de .github/workflows/, y configurará los secretos necesarios para la autenticación:
    - AZURE_CLIENT_ID
    - AZURE_TENANT_ID
    - AZURE_SUBSCRIPTION_ID
13. Abrir el repositorio en GitHub y entrar en **Actions**. Comprobar que el workflow se haya generado y que su ejecución finalice correctamente. A partir de entonces, los cambios enviados a la rama main podrán activar automáticamente el despliegue en Azure App Service, según la configuración del workflow.

Conectar Azure con GitHub Actions

Abrir el **Centro de implementación** de la aplicación. La conexión con GitHub permite generar el flujo en el repositorio y configurar la identidad que utilizará el despliegue (Microsoft, 2025b).

- En **Origen**, seleccionar **GitHub**.
- Mantener **GitHub Actions** como proveedor.
- Autorizar GitHub y seleccionar la cuenta propietaria del repositorio. En el ejemplo es artemioadaysolvers; en una copia individual será la cuenta del estudiante.
- Seleccionar Tutorial-Web-I y la rama main.
- Si se solicitan la pila y la versión, seleccionar las configuradas en la aplicación: Node - 26.
- En autenticación, seleccionar **Identidad asignada por el usuario**; crear o elegir la propuesta por Azure.
- Pulsar **Guardar**.

Referencias

Adobe. (2017, 25 de julio). *Flash & the future of interactive content.* Adobe Blog. https://blog.adobe.com/adobe-flash-update/

Adobe. (2021). *Adobe Flash Player EOL general information.* https://www.adobe.com/products/flashplayer/end-of-life.html

Apple. (s. f.-a). *AppKit*. Apple Developer Documentation. https://developer.apple.com/documentation/appkit

Apple. (s. f.-b). *SwiftUI*. Apple Developer Documentation. https://developer.apple.com/documentation/swiftui

artemioadaysolvers. (s. f.-a). *AutoMind-URDF-USD-MJCF-Loader* [Código fuente]. GitHub. https://github.com/artemioadaysolvers/AutoMind-URDF-USD-MJCF-Loader

artemioadaysolvers. (s. f.-b). *Tutorial-Web-I* [Código fuente]. GitHub. Recuperado el 20 de octubre de 2026, de https://github.com/artemioadaysolvers/Tutorial-Web-I

Canonical. (s. f.-a). *Basic installation*. Ubuntu Server Documentation. https://ubuntu.com/server/docs/tutorial/basic-installation/

Canonical. (s. f.-b). *How to set up SSSD with Active Directory*. Ubuntu Server documentation. https://ubuntu.com/server/docs/how-to/sssd/with-active-directory/

Canonical. (s. f.-c). *OpenSSH server*. Ubuntu Server documentation. https://ubuntu.com/server/docs/how-to/security/openssh-server/

Canonical. (s. f.-d). *User management*. Ubuntu Server documentation. https://ubuntu.com/server/docs/how-to/security/user-management/

Canonical. (s. f.-e). *Web services*. Ubuntu Server Documentation. https://ubuntu.com/server/docs/how-to/web-services/

Centrum Wiskunde & Informatica. (2015). *25 years of Python at CWI*. https://www.cwi.nl/en/news/25-years-of-python-at-cwi/

CERN. (s. f.). *The birth of the Web*.
https://home.cern/science/computing/the-birth-of-the-web

Chacon, S. (2009, 2 de enero). *Only the beginning*. The GitHub Blog.
https://github.blog/news-insights/the-library/only-the-beginning/

Debian Project. (2022). *Debian social contract (version 1.2)*.
https://www.debian.org/social_contract.en.html

Do, T. N. (2026). *Zig computation benchmark*. Sharkbench.
https://sharkbench.dev/computation/zig

Docker. (s. f.-a). *Docker Engine security*. Docker Docs.
https://docs.docker.com/engine/security/

Docker. (s. f.-b). *What is Docker?* Docker Docs.
https://docs.docker.com/get-started/docker-overview/

Ecma International. (2025, junio). *ECMAScript 2025 language specification* (ECMA-262, 16.ª ed.).
https://262.ecma-international.org/16.0/index.html

Fielding, R., Nottingham, M., & Reschke, J. (Eds.). (2022, junio). *HTTP semantics* (RFC 9110). Internet Engineering Task Force.
https://www.rfc-editor.org/rfc/rfc9110

Ginger Bill. (s. f.). *Overview: Odin programming language*.
https://odin-lang.org/docs/overview/

GitHub. (s. f.-a). *About GitHub Desktop*. GitHub Docs.
https://docs.github.com/en/desktop/overview/about-github-desktop

GitHub. (s. f.-b). *Acerca de los pull requests*. GitHub Docs. https://docs.github.com/es/pull-requests/get-started/about-pull-requests

GitHub. (s. f.-c). *Configuración de OpenID Connect en Azure*. GitHub Docs.
https://docs.github.com/es/actions/how-tos/secure-your-work/security-harden-deployments/oidc-in-azure

GitHub. (s. f.-d). *GitHub-hosted runners reference*. GitHub Docs. https://docs.github.com/en/actions/reference/runners/github-hosted-runners

GitHub. (s. f.-e). *Hello world*. GitHub Docs. https://docs.github.com/en/get-started/using-github/hello-world

GitHub. (s. f.-f). *Managing your personal access tokens*. GitHub Docs. https://docs.github.com/en/authentication/keeping-your-account-and-data-secure/managing-your-personal-access-tokens

GitHub. (s. f.-g). *¿Qué es GitHub?*. GitHub Docs. https://docs.github.com/es/get-started/start-your-journey/what-is-github

GitHub. (s. f.-h). *Self-hosted runners*. GitHub Docs. https://docs.github.com/en/actions/concepts/runners/self-hosted-runners

GitHub. (s. f.-i). *Understanding GitHub Actions*. GitHub Docs. https://docs.github.com/en/actions/get-started/understand-github-actions

GlebOlsen. (2026). *benchLANG: Programming language benchmark 2026* [Repositorio de código]. GitHub. https://github.com/GlebOlsen/benchLANG

Gomes, J. P., Monteiro, R., Moreira, A. S., Mendonça, V., Pinho, A., & Cunha, C. R. (2026). Teaching introductory programming in top universities: A global study of languages, paradigms, assessment, and AI. *Journal of Computer Languages, 86*, 101384. https://doi.org/10.1016/j.cola.2025.101384

Google. (s. f.). *Develop UI*. Android Developers. https://developer.android.com/develop/ui

GTK Development Team. (s. f.). *Getting started with GTK*. GTK 4 Documentation. https://docs.gtk.org/gtk4/getting_started.html

Horn, R., Lahnaoui, A., Reinoso, E., Peng, S., Isakov, V., Islam, T., & Malavolta, I. (2023). Native vs web apps: Comparing the energy consumption and performance of Android apps and their web counterparts. En *2023 IEEE/ACM 10th International Conference on Mobile Software Engineering and Systems (MOBILESoft)* (pp. 44-54). IEEE. https://doi.org/10.1109/MOBILSoft59058.2023.00013

International Digital Economy Academy. (s. f.). *MoonBit*. https://www.moonbitlang.com/

Internet Security Research Group. (s. f.). *How it works*. Let's Encrypt. https://letsencrypt.org/how-it-works/

Jullien, N., Clément-Fontaine, M., & Dalle, J.-M. (Coords.). (2002). *Nouveaux modèles économiques, nouvelle économie du logiciel* [Informe final de investigación]. Projet RNTL. https://www.marsouin.org/IMG/pdf/fichier_rapport.pdf

Khronos Group. (s. f.). *WebGL: 3D graphics for the Web*. https://www.khronos.org/webgl/

LaForge, A. (2016, 9 de agosto). *Flash and Chrome*. Google Blog. https://blog.google/products-and-platforms/products/chrome/flash-and-chrome/

Linux Kernel Development Community. (s. f.). *KVM*. The Linux Kernel documentation. https://docs.kernel.org/virt/kvm/index.html

MariuzM. (s. f.). *Bench: Head-to-head programming language benchmarks* [Repositorio de código]. GitHub. https://github.com/MariuzM/bench

MDN Web Docs. (s. f.-a). *HTML: HyperText Markup Language*. https://developer.mozilla.org/en-US/docs/Web/HTML

MDN Web Docs. (s. f.-b). *JavaScript*. https://developer.mozilla.org/en-US/docs/Glossary/JavaScript

MDN Web Docs. (s. f.-c). *What is a progressive web app?* Mozilla. https://developer.mozilla.org/en-US/docs/Web/Progressive_web_apps/Guides/What_is_a_progressive_web_app

Mernik, M., Heering, J., & Sloane, A. M. (2005). When and how to develop domain-specific languages. *ACM Computing Surveys, 37*(4), 316-344. https://doi.org/10.1145/1118890.1118892

Metri, G., Shi, W., & Brockmeyer, M. (2015). Energy-efficiency comparison of mobile platforms and applications: A quantitative approach. En *Proceedings of the 16th International Workshop on Mobile Computing Systems and Applications* (pp. 39-44). Association for Computing Machinery. https://doi.org/10.1145/2699343.2699358

Meyerovich, L. A., & Rabkin, A. S. (2013). Empirical analysis of programming language adoption. *ACM SIGPLAN Notices, 48*(10), 1-18. https://doi.org/10.1145/2509136.2509515

Microsoft. (s. f.-a). *Active Directory Domain Services overview*. Microsoft Learn. https://learn.microsoft.com/en-us/windows-server/identity/ad-ds/get-started/virtual-dc/active-directory-domain-services-overview

Microsoft. (s. f.-b). *Azure Functions Host* [Código fuente]. GitHub. https://github.com/Azure/azure-functions-host

Microsoft. (s. f.-c). *Azure Hybrid Benefit for Windows Server*. Microsoft Learn. https://learn.microsoft.com/en-us/windows-server/get-started/azure-hybrid-benefit

Microsoft. (s. f.-d). *C# language documentation*. Microsoft Learn. https://learn.microsoft.com/en-us/dotnet/csharp/

Microsoft. (s. f.-e). *Componentes y límites del sistema de información de Azure*. Microsoft Learn. https://learn.microsoft.com/es-es/azure/security/fundamentals/infrastructure-components

Microsoft. (s. f.-f). *Hyper-V virtualization in Windows Server and Windows*. Microsoft Learn. https://learn.microsoft.com/en-us/windows-server/virtualization/hyper-v/overview

Microsoft. (s. f.-g). *Install the Web Server WEB1*. Microsoft Learn. https://learn.microsoft.com/en-us/windows-server/networking/core-network-guide/cncg/server-certs/install-the-web-server-web1

Microsoft. (s. f.-h). *Introduction to .NET*. Microsoft Learn. https://learn.microsoft.com/en-us/dotnet/core/introduction

Microsoft. (s. f.-i). *Linux virtual machines pricing*. Microsoft Azure. https://azure.microsoft.com/en-us/pricing/details/virtual-machines/linux/

Microsoft. (s. f.-j). *Manage system node pools in Azure Kubernetes Service (AKS)*. Microsoft Learn. https://learn.microsoft.com/en-us/azure/aks/use-system-pools

Microsoft. (s. f.-k). *¿Qué es Azure?*. Microsoft Azure. https://azure.microsoft.com/es-es/resources/cloud-computing-dictionary/what-is-azure/

Microsoft. (s. f.-l). *¿Qué es Azure Linux Container Host para Azure Kubernetes Service (AKS)?*. Microsoft Learn. https://learn.microsoft.com/es-es/azure/azure-linux/azure-linux-aks-overview

Microsoft. (s. f.-m). *¿Qué es Windows Server?*. Microsoft Learn. https://learn.microsoft.com/es-es/windows-server/get-started/overview

Microsoft. (s. f.-n). *¿Qué son las regiones de Azure?*. Microsoft Learn. https://learn.microsoft.com/es-es/azure/reliability/regions-overview

Microsoft. (s. f.-o). *The history of C#*. Microsoft Learn. https://learn.microsoft.com/en-us/dotnet/csharp/whats-new/csharp-version-history

Microsoft. (s. f.-p). *The TypeScript handbook*. TypeScript Documentation. https://www.typescriptlang.org/docs/handbook/intro.html

Microsoft. (s. f.-q). *Use the Azure Login action with OpenID Connect*. Microsoft Learn. https://learn.microsoft.com/en-us/azure/developer/github/connect-from-azure-openid-connect

Microsoft. (s. f.-r). *Virtual machines in Azure*. Microsoft Learn. https://learn.microsoft.com/en-us/azure/virtual-machines/overview

Microsoft. (s. f.-s). *Windows authentication overview*. Microsoft Learn. https://learn.microsoft.com/en-us/windows-server/security/windows-authentication/windows-authentication-overview

Microsoft. (s. f.-t). *Windows container version compatibility*. Microsoft Learn. https://learn.microsoft.com/en-us/virtualization/windowscontainers/deploy-containers/version-compatibility

Microsoft. (s. f.-u). *Windows developer glossary*. Microsoft Learn. https://learn.microsoft.com/en-us/windows/apps/get-started/windows-developer-glossary

Microsoft. (s. f.-v). *Windows Server pricing and licensing*. https://www.microsoft.com/en-us/windows-server/pricing

Microsoft. (2000, 26 de junio). *Microsoft introduces highly productive .NET programming language: C#*. Microsoft News. https://news.microsoft.com/source/2000/06/26/microsoft-introduces-highly-productive-net-programming-language-c/

Microsoft. (2008, 27 de octubre). *Microsoft unveils Windows Azure at Professional Developers Conference*. Microsoft News. https://news.microsoft.com/source/2008/10/27/microsoft-unveils-windows-azure-at-professional-developers-conference/

Microsoft. (2010, 1 de febrero). *Windows Azure general availability*. https://blogs.microsoft.com/blog/2010/02/01/windows-azure-general-availability/

Microsoft. (2022). *Ten years of TypeScript*. Microsoft DevBlogs. https://devblogs.microsoft.com/typescript/ten-years-of-typescript/

Microsoft. (2023a, 14 de noviembre). *Estrategias de arquitectura para proteger los secretos de aplicación*. Microsoft Learn. https://learn.microsoft.com/es-es/azure/well-architected/security/application-secrets

Microsoft. (2023b). *Overview of .NET Framework*. Microsoft Learn. https://learn.microsoft.com/en-us/dotnet/framework/get-started/overview

Microsoft. (2024, 26 de junio). *¿Qué es Azure Static Web Apps?*. Microsoft Learn. https://learn.microsoft.com/es-es/azure/static-web-apps/overview

Microsoft. (2025a, 14 de septiembre). *Configure a PHP app in Azure App Service*. Microsoft Learn. https://learn.microsoft.com/en-us/azure/app-service/configure-language-php

Microsoft. (2025b, 16 de junio). *Configure continuous deployment to Azure App Service*. Microsoft Learn. https://learn.microsoft.com/en-us/azure/app-service/deploy-continuous-deployment

Microsoft. (2025c, 1 de mayo). *Create a PHP web app in Azure App Service*. Microsoft Learn. https://learn.microsoft.com/en-us/azure/app-service/quickstart-php

Microsoft. (2025d, 19 de septiembre). *Estilo de arquitectura de n niveles*. Microsoft Learn. https://learn.microsoft.com/es-es/azure/architecture/guide/architecture-styles/n-tier

Microsoft. (2025e, 28 de noviembre). *Implementación en Azure App Service mediante Acciones de GitHub*. Microsoft Learn. https://learn.microsoft.com/es-es/azure/app-service/deploy-github-actions

Microsoft. (2025f, 3 de abril). *Procedimientos recomendados para el diseño de la API web RESTful*. Microsoft Learn. https://learn.microsoft.com/es-es/azure/architecture/best-practices/api-design

Microsoft. (2025g, 31 de octubre). *What is the Server Core installation option in Windows Server?*. Microsoft Learn. https://learn.microsoft.com/en-us/windows-server/administration/server-core/what-is-server-core

Microsoft. (2026a, 25 de septiembre). *Idiomas admitidos en Azure Functions*. Microsoft Learn. https://learn.microsoft.com/es-es/azure/azure-functions/supported-languages

Microsoft. (2026b, 18 de agosto). *Overview of Azure App Service*. Microsoft Learn. https://learn.microsoft.com/en-us/azure/app-service/overview

Microsoft. (2026c, 18 de septiembre). *¿Qué es Azure Functions?*. Microsoft Learn. https://learn.microsoft.com/es-es/azure/azure-functions/functions-overview

Microsoft. (2026d, 24 de agosto). *What are Azure App Service plans?* Microsoft Learn. https://learn.microsoft.com/en-us/azure/app-service/overview-hosting-plans

Milichiewicz, R., & Badurowicz, M. (2025). Comparative analysis of cross-platform application development tools in terms of operating system integration. *Journal of Computer Sciences Institute, 36*, 357–364. https://doi.org/10.35784/jcsi.7831

MoonBit. (s. f.). *WebAssembly integration*. MoonBit Documentation. https://docs.moonbitlang.com/en/latest/toolchain/wasm/

MoonBit. (2024). *MoonBit adds JS backend, up to 25x faster than native JS*. MoonBit Blog. https://www.moonbitlang.com/blog/js-support

Nakashima, J., Atassi, H., & Thorpe, D. (2010, abril). *Desarrollo e implementación de aplicaciones de Windows Azure en Visual Studio 2010*. MSDN Magazine. https://learn.microsoft.com/es-es/archive/msdn-magazine/2010/april/microsoft-azure-developing-and-deploying-azure-apps-in-visual-studio-2010

Nim Contributors. (s. f.). *Nim programming language*. https://nim-lang.org/

Nim Development Team. (s. f.). *Nim compiler user guide*. https://nim-lang.org/docs/nimc.html

Nisan, N., & Schocken, S. (2005). *The elements of computing systems: Building a modern computer from first principles*. MIT Press. https://mitpress.mit.edu/9780262640688/the-elements-of-computing-systems/

OpenJS Foundation. (s. f.). *Don't block the event loop (or the worker pool)*. Node.js. https://nodejs.org/en/learn/asynchronous-work/dont-block-the-event-loop

OWASP Foundation. (s. f.). *Secure product design cheat sheet*. https://cheatsheetseries.owasp.org/cheatsheets/Secure_Product_Design_Cheat_Sheet.html

Pereira, R., Couto, M., Ribeiro, F., Rua, R., Cunha, J., Fernandes, J. P., & Saraiva, J. (2017a). Energy efficiency across programming languages: How do energy, time, and memory relate? En *Proceedings of the 10th ACM SIGPLAN International Conference on Software Language Engineering* (pp. 256-267). Association for Computing Machinery. https://doi.org/10.1145/3136014.3136031

Pereira, R., Couto, M., Ribeiro, F., Rua, R., Cunha, J., Fernandes, J. P., & Saraiva, J. (2017b). Energy efficiency across programming languages: How do energy, time, and memory relate? En *Proceedings of the 10th ACM SIGPLAN International Conference on Software Language Engineering* (pp. 256-267). ACM. https://greenlab.di.uminho.pt/wp-content/uploads/2017/10/sleFinal.pdf

Python Software Foundation. (s. f.-a). *Design and history FAQ (Python 3.12)*. https://docs.python.org/3.12/faq/design.html

Python Software Foundation. (s. f.-b). *The Python tutorial*. Python Documentation. https://docs.python.org/3/tutorial/

Python Software Foundation. (s. f.-c). *What is Python? Executive summary*. https://www.python.org/doc/essays/blurb/

Ragan-Kelley, J., Barnes, C., Adams, A., Paris, S., Durand, F., & Amarasinghe, S. (2013). Halide: A language and compiler for optimizing parallelism, locality, and recomputation in image processing pipelines. En *Proceedings of the 34th ACM SIGPLAN Conference on Programming Language Design and Implementation* (pp. 519-530). ACM. https://people.csail.mit.edu/jrk/halide-pldi13.pdf

Rescorla, E. (2000, mayo). *HTTP over TLS* (RFC 2818). Internet Engineering Task Force. https://www.rfc-editor.org/rfc/rfc2818

Rescorla, E. (2018, agosto). *The Transport Layer Security (TLS) protocol version 1.3* (RFC 8446). Internet Engineering Task Force. https://www.rfc-editor.org/rfc/rfc8446

Shrestha, N., Botta, C., Barik, T., & Parnin, C. (2020). Here we go again: Why is it difficult for developers to learn another programming language? En *Proceedings of the ACM/IEEE 42nd International Conference on Software Engineering* (pp. 691-701). ACM. https://doi.org/10.1145/3377811.3380352

SixtyFPS GmbH. (s. f.). *Slint*. https://slint.dev/

Slint. (s. f.). *Web*. Slint Documentation. https://docs.slint.dev/latest/docs/slint/guide/platforms/web/

Szekeres, L., Payer, M., Wei, T., & Song, D. (2013). SoK: Eternal war in memory. En *2013 IEEE Symposium on Security and Privacy* (pp. 48-62). IEEE. https://doi.org/10.1109/SP.2013.13

Tanenbaum, A. S., & Wetherall, D. J. (2012). *Redes de computadoras* (5.ª ed.). Pearson Educación.

The Qt Company. (s. f.). *All overviews*. Qt for Python Documentation. https://doc.qt.io/qtforpython-6/overviews/qtdoc-overviews.html

Three.js. (s. f.). *WebGLRenderer*. Three.js Docs. https://threejs.org/docs/pages/WebGLRenderer

Torano, E. (2017). *Faster command line tools in Nim*. Nim Blog. https://nim-lang.org/blog/2017/05/25/faster-command-line-tools-in-nim.html

W3C. (s. f.). *A brief history of CSS until 2016*. https://www.w3.org/Style/CSS20/history

Wanstrath, C. (2008, 10 de abril). *We launched*. The GitHub Blog. https://github.blog/news-insights/we-launched/

WebKit Project. (2017, 25 de julio). *Adobe announces Flash distribution and updates to end*. WebKit. https://webkit.org/blog/7839/adobe-announces-flash-distribution-and-updates-to-end/

WHATWG. (2026, 19 de octubre). *HTML: Living standard*. https://html.spec.whatwg.org/multipage/

Wirfs-Brock, A., & Eich, B. (2020). JavaScript: The first 20 years. *Proceedings of the ACM on Programming Languages, 4*(HOPL), artículo 77, 1-189. https://wirfs-brock.com/allen/posts/866

World Wide Web Consortium. (2014). *HTML5: A vocabulary and associated APIs for HTML and XHTML*. W3C Recommendation. https://www.w3.org/TR/2014/REC-html5-20141028/

Zig Software Foundation. (s. f.). *Overview: Zig programming language*. https://ziglang.org/learn/overview/